4.3 Implementasi dengan Pustaka Python

4.3.1 NLTK

NLTK menyediakan POS tagger berbasis Penn Treebank dan NER tagger sederhana.

```
import nltk
from nltk import word_tokenize, pos_tag, ne_chunk

text = "Barack Obama was born in Hawaii."
tokens = word_tokenize(text)
tagged = pos_tag(tokens)
# [('Barack', 'NNP'), ('Obama', 'NNP'), ('was', 'VBD'), ...]

entities = ne_chunk(tagged)
# Tree dengan node berlabel PERSON, GPE, dst.
```

4.3.2 spaCy

spaCy lebih cepat, akurat, dan production-ready dibanding NLTK. Mendukung POS tagging dan NER dalam satu pipeline.

```
import spacy
nlp = spacy.load('en_core_web_sm')  # atau 'xx_ent_wiki_sm' untuk multibahasa

doc = nlp("Apple is looking at buying U.K. startup for $1 billion")
for token in doc:
    print(token.text, token.pos_, token.tag_)

for ent in doc.ents:
    print(ent.text, ent.label_)
# Apple ORG, U.K. GPE, $1 billion MONEY
```

4.3.3 Perbandingan NLTK vs spaCy

Aspek	NLTK	spaCy
Tujuan	Edukasi dan riset	Produksi
Kecepatan	Lambat	Sangat cepat (Cython)
Akurasi default	Cukup	Tinggi (model neural)
Multi-language	Manual	Banyak built-in models
API	Modular, prosedural	Pipeline objek-oriented

Bab 5 — Klasifikasi Teks

Klasifikasi teks (text classification) adalah task supervised learning untuk menetapkan satu atau lebih label kelas pada dokumen teks. Aplikasinya luas: analisis sentimen, deteksi spam, kategorisasi berita, deteksi hate speech, dan pemfilteran konten.

5.1 Alur Pipeline Klasifikasi Teks

Berikut urutan kerja standar yang harus dihafalkan dan dipahami.

11. **Pengumpulan Data —** kumpulkan dataset berlabel yang merepresentasikan distribusi target. Pastikan kelas seimbang atau lakukan teknik balancing.
12. **Pra-pemrosesan —** lakukan tokenisasi, lowercasing, penghapusan stopword, stemming, dan pembersihan karakter khusus (lihat Bab 1).
13. **Ekstraksi Fitur —** ubah teks menjadi vektor numerik menggunakan BoW, TF-IDF, n-gram, atau embedding (Word2Vec, GloVe, BERT) (lihat Bab 2).
14. **Pelatihan Model —** split data menjadi train/validation/test (umumnya 70/15/15 atau 80/10/10), pilih algoritma, dan latih model dengan tuning hyperparameter.
15. **Evaluasi —** ukur performa pada data uji menggunakan accuracy, precision, recall, F1-score, dan confusion matrix.
16. **Prediksi —** deploy model untuk inferensi pada data baru, dengan pra-pemrosesan dan vectorization yang konsisten dengan tahap training.

5.2 Algoritma: ML Tradisional vs Deep Learning

5.2.1 Algoritma Machine Learning Tradisional

a. Naive Bayes

Berbasis Bayes Theorem dengan asumsi naif bahwa fitur (kata) saling independen. Sangat cepat, baseline kuat untuk klasifikasi teks, terutama dengan TF-IDF.

$$P(c|d) \propto P(c) \times \Pi\, P(w_i|c)$$

- **Varian:** Multinomial NB (untuk count features), Bernoulli NB (untuk binary features).

b. Logistic Regression

Model linier yang memprediksi probabilitas kelas menggunakan fungsi sigmoid (binary) atau softmax (multi-class). Stabil, mudah diinterpretasi, dan menjadi baseline yang kuat ketika dipasangkan dengan TF-IDF.

c. Support Vector Machine (SVM)

Mencari hyperplane optimal yang memisahkan kelas dengan margin terbesar. Sangat efektif pada ruang fitur berdimensi tinggi seperti TF-IDF text features.

- **Linear SVM:** default untuk text classification, cepat dan akurat.
- **Kernel SVM (RBF, poly):** untuk batas keputusan non-linier; jarang dibutuhkan pada teks.

5.2.2 Algoritma Deep Learning

a. RNN dan LSTM/GRU

Recurrent Neural Network memproses teks secara sekuensial, mempertahankan hidden state yang merepresentasikan konteks. LSTM dan GRU mengatasi masalah vanishing gradient dengan gating mechanism.

- Cocok untuk task yang memerlukan dependensi temporal jangka panjang.
- Bidirectional LSTM (BiLSTM) memproses dari kiri-ke-kanan dan kanan-ke-kiri untuk konteks penuh.

b. CNN untuk Teks

Convolutional Neural Network dengan filter 1D yang menyapu kata-kata, efektif menangkap fitur lokal seperti n-gram secara otomatis. Cepat dilatih dan kompetitif dengan RNN pada banyak task.

c. Transformer dan BERT

Arsitektur Transformer dengan mekanisme self-attention menjadi state-of-the-art. BERT (Bidirectional Encoder Representations from Transformers) di-pretrain pada korpus besar dan di-fine-tune untuk task downstream.

- **Variants:** BERT, RoBERTa, DistilBERT, ALBERT, dan untuk Bahasa Indonesia: IndoBERT, IndoBERTweet.
- **Fine-tuning:** tambahkan classification head di atas [CLS] token dan latih pada data domain.

5.2.3 Perbandingan ML Tradisional vs Deep Learning

Aspek	ML Tradisional	Deep Learning
Kebutuhan Data	Sedikit hingga sedang	Banyak (atau pre-trained)
Kebutuhan Compute	Rendah (CPU)	Tinggi (GPU/TPU)
Feature Engineering	Manual (TF-IDF, n-gram)	Otomatis (learned embeddings)
Interpretasi	Tinggi	Rendah (black box)
Performa	Baik untuk task sederhana	SOTA untuk task kompleks
Waktu Inferensi	Sangat cepat	Lebih lambat (kecuali distilled)

5.3 Evaluasi

5.3.1 Confusion Matrix

Confusion matrix adalah tabel yang merangkum hasil prediksi pada data uji untuk klasifikasi biner.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

5.3.2 Metrik Evaluasi

Accuracy — proporsi prediksi yang benar dari total prediksi.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

Catatan: accuracy menyesatkan pada kelas yang tidak seimbang. Pada dataset 95% negatif, model yang selalu memprediksi negatif tetap mendapat accuracy 95%.

Precision — dari semua yang diprediksi positif, berapa yang benar-benar positif.

$$Precision = TP / (TP + FP)$$

Penting saat false positive mahal, contoh: filter spam (jangan sampai email penting masuk spam).

Recall (Sensitivity) — dari semua yang sebenarnya positif, berapa yang berhasil ditemukan.

$$Recall = TP / (TP + FN)$$

Penting saat false negative mahal, contoh: deteksi penyakit (jangan sampai pasien sakit terdeteksi sehat).

F1-Score — harmonic mean dari precision dan recall, memberikan satu metrik yang menyeimbangkan keduanya.

$$F1 = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Gunakan F1 sebagai metrik utama saat kelas tidak seimbang dan kedua precision-recall sama pentingnya.

5.3.3 Metrik untuk Multi-class

- **Macro-average:** hitung metrik per kelas lalu rata-ratakan tanpa pembobotan. Semua kelas dianggap setara.
- **Micro-average:** agregasi TP, FP, FN dari semua kelas dulu, baru hitung metrik. Didominasi kelas mayoritas.
- **Weighted-average:** rata-rata dengan bobot proporsional jumlah sampel per kelas.

5.3.4 Contoh Implementasi

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, digits=4))
```

Bab 6 — Pemodelan Topik

Topic Modeling adalah teknik unsupervised untuk menemukan struktur tematik tersembunyi dalam koleksi dokumen. Setiap dokumen direpresentasikan sebagai campuran topik, dan setiap topik direpresentasikan sebagai distribusi kata.

6.1 LDA (Latent Dirichlet Allocation)

LDA adalah model probabilistik generatif yang mengasumsikan dokumen dihasilkan dari proses berikut: pilih distribusi topik untuk dokumen, lalu untuk setiap kata pilih topik dari distribusi itu, lalu pilih kata dari distribusi kata di topik tersebut.

6.1.1 Asumsi Generatif

17. Setiap dokumen adalah campuran beberapa topik dengan proporsi tertentu.
18. Setiap topik adalah distribusi probabilitas atas vocabulary.
19. Distribusi proporsi topik per dokumen mengikuti Dirichlet dengan parameter α.
20. Distribusi kata per topik mengikuti Dirichlet dengan parameter β.

6.1.1 Asumsi Generatif

17. Setiap dokumen adalah campuran beberapa topik dengan proporsi tertentu.
18. Setiap topik adalah distribusi probabilitas atas vocabulary.
19. Distribusi proporsi topik per dokumen mengikuti Dirichlet dengan parameter α.
20. Distribusi kata per topik mengikuti Dirichlet dengan parameter β.

6.1.2 Hyperparameter

Parameter	Pengaruh	Interpretasi
α (alpha)	Distribusi topik per dokumen	α tinggi → dokumen memuat banyak topik; α rendah → dokumen fokus pada sedikit topik
β (beta / eta)	Distribusi kata per topik	β tinggi → topik berisi banyak kata berbeda; β rendah → topik fokus pada sedikit kata kunci
K	Jumlah topik	Hyperparameter yang harus ditentukan; pilih dengan coherence score

6.1.3 Inferensi

Karena posterior eksak intractable, LDA menggunakan metode aproksimasi:

- **Variational Inference:** optimasi parameter variational distribution untuk mendekati posterior.
- **Collapsed Gibbs Sampling:** MCMC sampling untuk mengestimasi distribusi topik.

6.1.4 Implementasi

```
from gensim import corpora, models

dictionary = corpora.Dictionary(processed_docs)
corpus = [dictionary.doc2bow(doc) for doc in processed_docs]

lda = models.LdaModel(corpus, num_topics=10, id2word=dictionary,
                      alpha='auto', eta='auto', passes=20)

for idx, topic in lda.print_topics(-1):
    print(f'Topic {idx}: {topic}')
```

6.2 LSA (Latent Semantic Analysis)

LSA adalah pendekatan aljabar linear yang menemukan struktur laten dalam matriks term-document menggunakan Singular Value Decomposition (SVD). LSA mengurangi dimensi dan menangkap relasi semantik antar kata yang sering muncul bersama.

6.2.1 Mekanisme SVD

Matriks term-document A (m×n, m=jumlah term, n=jumlah dokumen) didekomposisi menjadi:

$$A = U \times \Sigma \times V^T$$

- **U** (m×k): matriks term-topic, merepresentasikan term dalam ruang topik.
- **Σ** (k×k): matriks diagonal singular values, urutan kepentingan topik.
- **Vᵀ** (k×n): matriks topic-document, merepresentasikan dokumen dalam ruang topik.

Dengan mengambil k singular values terbesar (truncated SVD), kita memperoleh representasi dimensi rendah yang menangkap struktur semantik utama.

6.2.2 Implementasi

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

tfidf = TfidfVectorizer(max_features=5000, stop_words='english')
X = tfidf.fit_transform(corpus)

lsa = TruncatedSVD(n_components=10, random_state=42)
X_lsa = lsa.fit_transform(X)
```

6.2.3 Kelebihan dan Kekurangan

- **Kelebihan:** matematika sederhana, dapat menangani sinonim hingga taraf tertentu, deterministik.
- **Kekurangan:** nilai pada matriks bisa negatif sehingga sulit diinterpretasi sebagai topik, tidak probabilistik, kurang fleksibel dibanding LDA.

6.3 NMF (Non-negative Matrix Factorization)

NMF adalah faktorisasi matriks dengan kendala bahwa semua entri matriks faktor harus non-negatif. Kendala ini menghasilkan representasi yang lebih mudah diinterpretasi dibanding SVD.

6.3.1 Formulasi

Matriks A (m×n, biasanya TF-IDF) didekomposisi menjadi dua matriks non-negatif W dan H:

$$A \approx W \times H, \quad W \geq 0,\ H \geq 0$$

- **W** (m×k): matriks bobot term-topic, kolom adalah topik dengan komposisi term.
- **H** (k×n): matriks bobot topic-document, kolom adalah dokumen dengan komposisi topik.

Optimasi dilakukan dengan meminimumkan $||A - WH||^2$ atau divergensi KL secara iteratif menggunakan multiplicative update rules.

6.3.2 Implementasi

```
from sklearn.decomposition import NMF

nmf = NMF(n_components=10, random_state=42, init='nndsvda')
W = nmf.fit_transform(X)  # bobot dokumen-topik
H = nmf.components_       # bobot topik-term

for i, topic in enumerate(H):
    top_words = [terms[j] for j in topic.argsort()[-10:][::-1]]
    print(f'Topic {i}: {top_words}')
```

6.3.3 NMF vs LDA vs LSA

Aspek	LDA	LSA	NMF
Pendekatan	Probabilistik	Aljabar linear (SVD)	Faktorisasi matriks
Non-negatif	Ya	Tidak	Ya
Interpretabilitas	Tinggi	Rendah	Tinggi
Stokastik	Ya	Tidak	Sebagian (init)
Cocok untuk korpus pendek	Kurang	Cukup	Baik

6.4 BERTopic

BERTopic adalah pendekatan modern yang menggabungkan kontekstual embeddings dari model Transformer (BERT atau Sentence-BERT) dengan teknik clustering untuk menghasilkan topik yang lebih koheren dan kontekstual.

6.4.1 Pipeline BERTopic

21. **Embedding:** hasilkan dense vector untuk setiap dokumen menggunakan Sentence-BERT atau model embedding lainnya.
22. **Pengurangan Dimensi (UMAP):** turunkan dimensi embedding (misal dari 768 ke 5–15) sambil mempertahankan struktur lokal dan global.
23. **Clustering (HDBSCAN):** kelompokkan embedding menjadi cluster berbasis density. HDBSCAN dapat mengidentifikasi outlier (noise) dan tidak memerlukan jumlah cluster predefined.
24. **Tokenization:** hitung representasi BoW per cluster.
25. **c-TF-IDF (class-based TF-IDF):** modifikasi TF-IDF di mana setiap cluster diperlakukan sebagai satu dokumen besar untuk mengekstrak kata kunci paling diskriminatif per topik.

6.4.2 Implementasi

```
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer

embedder = SentenceTransformer('paraphrase-multilingual-MiniLM-L12-v2')
topic_model = BERTopic(embedding_model=embedder, language='multilingual')

topics, probs = topic_model.fit_transform(documents)
topic_model.get_topic_info()
```

6.4.3 Kelebihan BERTopic

- Memanfaatkan kontekstualitas BERT, sehingga topik lebih koheren secara semantik.
- Tidak memerlukan jumlah topik predefined (HDBSCAN menentukan otomatis).
- Mendukung dynamic topic modeling, hierarchical topics, dan visualisasi interaktif bawaan.
- Bekerja baik untuk Bahasa Indonesia dengan multilingual embedding seperti IndoBERT atau LaBSE.

6.5 Evaluasi Model Topik

6.5.1 Perplexity

Perplexity mengukur seberapa baik model memprediksi sampel data baru. Pada topic modeling, dihitung dari log-likelihood dokumen uji.

$$Perplexity = exp(-\Sigma \log p(w) / N)$$

Interpretasi: nilai lebih kecil lebih baik. Namun perlu dicatat bahwa perplexity rendah tidak selalu berkorelasi dengan kualitas topik secara human judgment.

6.5.2 Topic Coherence

Topic coherence mengukur seberapa semantik koheren kata-kata teratas di suatu topik. Beberapa varian populer:

- **UMass:** berbasis log-conditional probability dari kata-kata top.
- **UCI:** berbasis pointwise mutual information (PMI) di sliding window.
- **CV:** kombinasi dari indirect cosine similarity dan NPMI; paling populer dan berkorelasi baik dengan human judgment.

Interpretasi: nilai mendekati 1 lebih baik untuk c_v dan c_npmi. Untuk umass, nilai mendekati 0 (sebenarnya negatif tapi mendekati nol) lebih baik.

```
from gensim.models import CoherenceModel

cm = CoherenceModel(model=lda, texts=docs, dictionary=dictionary, coherence='c_v')
print(cm.get_coherence())
```

6.5.3 Ringkasan Metrik

Metrik	Arah Lebih Baik	Catatan
Perplexity	Lebih kecil	Tidak selalu mencerminkan kualitas topik bagi manusia
Topic Coherence (c_v)	Mendekati 1	Berkorelasi baik dengan penilaian manusia, paling direkomendasikan
Topic Diversity	Mendekati 1	Mengukur keunikan kata antar topik (mengurangi redundansi)